

ARQUITECTURA CLIENTE-SERVIDOR Y PATRONES DE DESARROLLO WEB

DWEC



9 DE NOVIEMBRE DE 2026

DIEGO GÓMEZ ARCAS

INDICE

INTRODUCCION 2
1. Estructura del trabajo 3
1.1. Concepto general 3
Qué es una arquitectura cliente-servidor. 3
Diferencias respecto a una aplicación monolítica local. 3
Ventajas e inconvenientes. 3
1.2. Componentes principales 4
Cliente 4
Servidor 4
1.3. Comunicación entre cliente y servidor 5
Protocolo HTTP/HTTPS. 5
Peticiones y respuestas. 5
Métodos HTTP: 5
1.4. Ejemplo práctico 6
1.5. Modelo de capas 7
Capas habituales 7
Capa de Presentación 7
Capa de Acceso a Datos 7
1.6. Modelo - vista – controlador 8
Modelo 8
Vista 8
Controlador 8
Frameworks que utilizan MVC 8
1.7. API – REST 9
Principios REST 9
1.8 SPA (Single Page Application) 10
Frameworks 10
1.9 SSR (Server Side Rendering) 11
Frameworks 11
1.10 Arquitectura Jamstack 12
1. 11 Frontend 13
1. 12 Backend 14
Comparativa 14
2. Caso de estudio obligatorio 15
DIAGRAMA 16

INTRODUCCION

Este trabajo consiste en la búsqueda de información de Arquitectura Cliente-Servidor y Patrones de Desarrollo Web, en el informe explicamos al detalle cada punto que es preguntado.

En este trabajo se explica cómo funciona y qué patrones se usan para construir una web: capas, MVC, API REST, SPA, SSR y Jamstack.

1. Estructura del trabajo

1.1. Concepto general

Qué es una arquitectura cliente-servidor.

Es un modelo de diseño de software e informática donde las tareas y los recursos se reparten entre dos partes principales que son el cliente y el servidor.

El cliente es el dispositivo o la aplicación mientras que el servidor es el equipo o programa central.

Diferencias respecto a una aplicación monolítica local.

Las diferencias son que, en una aplicación monolítica local, la interfaz, la lógica y los datos viven en el mismo programa y en el mismo equipo. En la arquitectura cliente-servidor se reparten es decir el cliente presenta la información y el servidor la procesa y guarda los datos, comunicándose por red (HHTP/HTTPS).

Ventajas e inconvenientes.

Ventajas

- Datos centralizados y más fáciles de mantener.
- Actualizaciones en un solo sitio.
- Clientes ligeros: vale cualquier dispositivo con navegador.
- Seguridad y control de acceso centralizados.
- Cada componente se puede escalar por separado.

Inconvenientes

- El servidor es un punto único de fallo.
- Depende de la red.
- Puede saturarse con muchas peticiones.
- Más superficie de ataque.
- Más complejidad de diseño y operación.

1.2. Componentes principales

Cliente

Navegador web.

Es una aplicación instalada en el dispositivo del usuario que sirve de cliente para acceder a la web. Su función es pedir recursos a un servidor, interpretarlos y mostrarlos de forma visual.

Como lo interpreta, el HTML la estructura y el contenido luego el CSS es el aspecto y por último el JavaScript que es el comportamiento y la interacción.

Aplicaciones móviles.

Es un programa informático diseñado para funcionar en los dispositivos portátiles como teléfonos inteligentes y "tablets" con sistemas operativos como son Android e IOS.

Aplicaciones de escritorio.

Es un programa de software que se instala directamente en el sistema operativo de un ordenador como son Windows, macOS o Linux y se ejecuta de forma local utilizando recursos de hardware del equipo.

Servidor

Servidor web.

Es un sistema hardware y software para almacenar, procesar y entregar el contenido de los sitios web a los usuarios a través de internet.

Servidor de aplicaciones.

Es un programa o entorno de software que ejecuta la lógica de negocio, proceso datos dinámicos y conecta a los usuarios o clientes con las bases de datos y los recursos de red.

Base de datos.

Es un conjunto organizado de información estructurada que se almacena electrónicamente en un sistema informático para gestionar, consultar y actualizar datos de forma rápida y segura.

1.3. Comunicación entre cliente y servidor

Protocolo HTTP/HTTPS.

El HTTP (Protocolo de Transferencia de Hipertexto) y el HTTPS (Protocolo Seguro de Transferencia de Hipertexto) son las reglas que permiten la comunicación entre tu navegador web y los servidores donde están alojadas las páginas.

Peticiones y respuestas.

El modelo de petición-respuesta (request-response) es un esquema de comunicación en redes donde un dispositivo o cliente solicita un recurso a otro (el servidor), y este procesa la solicitud y devuelve un mensaje de respuesta.

Métodos HTTP:

GET

Se utiliza para solicitar y recuperar la representación de un recurso específico desde un servidor sin alterar su estado.

POST

Se utiliza para enviar datos a un servidor con el fin de crear un nuevo recurso o modificar uno existente.

PUT

Se utiliza para crear un nuevo recurso o reemplazar por completo una representación existente del recurso de destino con los datos que se envían en el cuerpo de la petición.

DELETE

Solicita la eliminación del recurso específico identificado por una URL en el servidor.

PATCH

Se utiliza para aplicar modificaciones parciales a un recurso existente, enviando en el cuerpo de la petición únicamente a los campos o atributos que se desean cambiar, en lugar de reemplazar el objeto completo.

1.4. Ejemplo práctico

Ejemplo práctico: consultar un producto en una tienda online

El usuario abre https://www.tienda-ejemplo.es/producto/42 en su navegador.

DNS: el navegador traduce www.tienda-ejemplo.es a una dirección IP.

Conexión segura: abre una conexión TCP y negocia TLS (HTTPS).

Petición: envía al servidor:

- GET /api/productos/42 HTTP/1.1
- Host: www.tienda-ejemplo.es
- Accept: application/json

Servidor web (Nginx): recibe la petición y la reenvía al backend.

Backend: valida el dato y consulta la base de datos:

sql

```
SELECT * FROM producto WHERE id = 42;
```

Respuesta: el backend devuelve 200 OK con un JSON:

json

```
{ "id": 42, "nombre": "Auriculares BT", "precio": 39.90, "stock": 12 }
```

Navegador: interpreta el JSON con JavaScript y dibuja la ficha del producto (nombre, precio, disponibilidad).

Si el producto no existiera, el servidor respondería 404 Not Found y el navegador mostraría un mensaje de error.

1.5. Modelo de capas

Qué es una arquitectura en capas.

Es un patrón de diseño de software que organiza el código de una aplicación en capas horizontales, donde cada una tiene una responsabilidad única y bien defendida.

Objetivos.

Dividir un sistema en partes más pequeñas, ordenadas y con responsabilidades, especificas para facilitar su diseño, comprensión y mantenimiento.

Ventajas.

Divide un sistema complejo en niveles horizontales independientes donde cada uno cumple una función específica.

Capas habituales

Capa de Presentación

Es la que guarda y recupera la información. Usa bases de datos como MySQL, PostgreSQL o MongoDB.

Capa de Lógica de Negocio

Es la que decide qué hacer con los datos: calcula precios, comprueba el stock y valida pedidos. Se programa en Java, PHP, C# o Node.js.

Capa de Acceso a Datos

Es la que guarda y recupera la información. Usa bases de datos como MySQL, PostgreSQL o MongoDB.

1.6. Modelo - vista – controlador

Modelo

Función.

Es un patrón de diseño de software que divide una aplicación en tres componentes interconectados para separar la lógica de negocio, los datos y la interfaz de usuario.

Gestión de datos.

La gestión de datos recae de forma exclusiva sobre el Modelo, que actúa como la capa encargada de conectar con la base de datos, aplicar las reglas del negocio y procesar la información.

Acceso a bases de datos.

El único componente que tiene permiso y responsabilidad para acceder directamente a la base de datos es el Modelo (a través de ORMs, repositorios o consultas SQL.

Vista

Representación visual.

Divide una aplicación en tres componentes principales interconectados para separar los datos, la interfaz visual y la lógica de control.

Interacción con el usuario.

Es el punto de partida de todo el flujo y es gestionada directamente por el Controlador.

Controlador

Recepción de peticiones.

Es la primera fase del flujo de la aplicación y la responsabilidad principal del Controlador (con la ayuda del sistema de enrutamiento).

Frameworks que utilizan MVC

Laravel

Es un framework de desarrollo web de código abierto, gratuito y basado en el lenguaje de programación PHP.

Spring MVC

Es un módulo del ecosistema Spring Framework que permite crear aplicaciones web y servicios RESTful en Java utilizando el patrón de diseño Modelo-Vista-Controlador (MVC).

ASP.NET MVC

Es un marco de trabajo (*framework*) de desarrollo web creado por Microsoft que implementa el patrón de arquitectura Modelo-Vista-Controlador (MVC)

Django (adaptación MTV)

Es una variación de la arquitectura tradicional MVC (Modelo-Vista-Controlador), donde el propio framework actúa como controlador.

1.7. API – REST

Qué es REST.

Es un estilo de arquitectura de software que define un conjunto de reglas para la comunicación entre sistemas a través de la web.

Qué es una API REST.

Es una interfaz de programación que permite la comunicación entre diferentes aplicaciones de software a través de internet, utilizando el protocolo HTTP.

Principios REST

Cliente-servidor.

La interfaz (cliente) y el almacenamiento de datos (servidor) están separados y evolucionan por su cuenta.

Stateless.

Cada petición lleva toda la información necesaria; el servidor no recuerda peticiones anteriores.

Caché.

Las respuestas indican si se pueden guardar para reutilizarlas y reducir la carga.

Interfaz uniforme.

Todos los recursos se tratan igual, con los mismos métodos HTTP (GET, POST, PUT, PATCH, DELETE).

Recursos identificados mediante URI.

El cliente no sabe si habla con el servidor final o con intermediarios (proxy, CDN, gateway).

1.8 SPA (Single Page Application)

Concepto y Funcionamiento.

Es un tipo de aplicación web o sitio que carga una única página HTML inicial y actualiza su contenido de forma dinámica (vía JavaScript y AJAX/APIs) a medida que el usuario interactúa con ella, sin recargar el navegador por completo

Frameworks

Angular

framework completo con TypeScript, de Google.

React

biblioteca de interfaces basada en componentes.

Vue

framework progresivo, de aprendizaje sencillo.

1.9 SSR (Server Side Rendering)

Es una técnica de desarrollo web en la que el servidor genera el código HTML completo de una página web cada vez que un usuario realiza una solicitud, enviando el contenido ya listo al navegador.

Ventajas.

Mejor SEO: los buscadores reciben el contenido ya renderizado.

Carga inicial más rápida: el usuario ve el primer contenido antes, sobre todo en móviles y redes lentas.

Inconvenientes.

Más carga en el servidor: debe generar el HTML en cada petición.

Más complejidad: el código tiene que funcionar tanto en el servidor como en el cliente.

Frameworks

Next.js

SSR para React.

Nuxt.js

SSR para Vue.

1.10 Arquitectura Jamstack

Concepto.

Es una arquitectura moderna de desarrollo web que desacopla el frontend del backend tradicional.

Casos de uso.

Es ideal para proyectos donde la prioridad es la velocidad de carga, la seguridad, el SEO y la escalabilidad global a través de una red de entrega de contenido (CDN).

1. 11 Frontend

Responsabilidades.

Es el responsable de construir la parte visual, interactiva y funcional de una aplicación web o móvil con la que los usuarios interactúan directamente en su navegador.

Tecnologías más utilizadas.

Los fundamentos Web: HTML, CSS y JavaScript.

Lenguaje Tipado: TypeScript.

Librerias y frameworks principales: React, Angular, Next.j s, vue.js y Svelte.

Estilos: Bootstrap.

1. 12 Backend

Es la parte de la aplicación que se ejecuta en el servidor y que el usuario no ve.

Responsabilidades.

- Aplicar la lógica de negocio.
- Exponer la API (REST) que usa el frontend.
- Guardar y consultar datos en la base de datos.
- Autenticar usuarios y proteger la aplicación de ataques.
- Conectarse con otros servicios (pagos, correo).
- Garantizar rendimiento y escalabilidad.

Tecnologías más utilizadas.

- Lenguajes y frameworks: Java (Spring Boot), PHP (Laravel), C# (ASP.NET), Node.js, Python (Django).
- Bases de datos: MySQL, PostgreSQL, MongoDB.
- Otras: Redis (caché) y Docker (contenedores).

Comparativa

Aspecto	Frontend	Backend
Objetivo	Ofrecer una interfaz usable, accesible y rápida	Procesar peticiones, aplicar la lógica y guardar los datos
Dónde se ejecuta	En el dispositivo del usuario (navegador o app)	En el servidor o en la nube
Tecnologías	HTML, CSS, JavaScript, React, Angular, Vue	Java (Spring Boot), PHP (Laravel), C#, Node.js, Python, MySQL,
Ejemplos	Carrito de la compra, formulario de pago, catálogo	Cálculo del total del pedido, control de stock, autenticación, API

2. Caso de estudio obligatorio

Spotify

Arquitectura cliente-servidor

- El cliente es la app de Spotify: en el móvil, el ordenador, la web, la tele o el coche.
- El servidor está en la nube y guarda las canciones, las listas y las cuentas.
- La app pide una canción y el servidor se la envía.

Posible uso de MVC

Spotify no cuenta cómo lo hace por dentro, pero es probable que use algo parecido:

- Modelo: los datos (canciones, listas, usuarios).
- Vista: lo que ves en la app.
- Controlador: recibe tu petición y busca los datos.

APIs utilizadas

- Tiene una API pública (REST) para consultar canciones, artistas y listas.
- La aplicación también usa APIs internas que no son públicas.
- Para iniciar sesión usa OAuth 2.0.

Frontend y backend

- Frontend: lo que ves y tocas (la app y la web).
- Backend: lo que hay detrás: buscar música, guardar listas, recomendar canciones y cobrar.

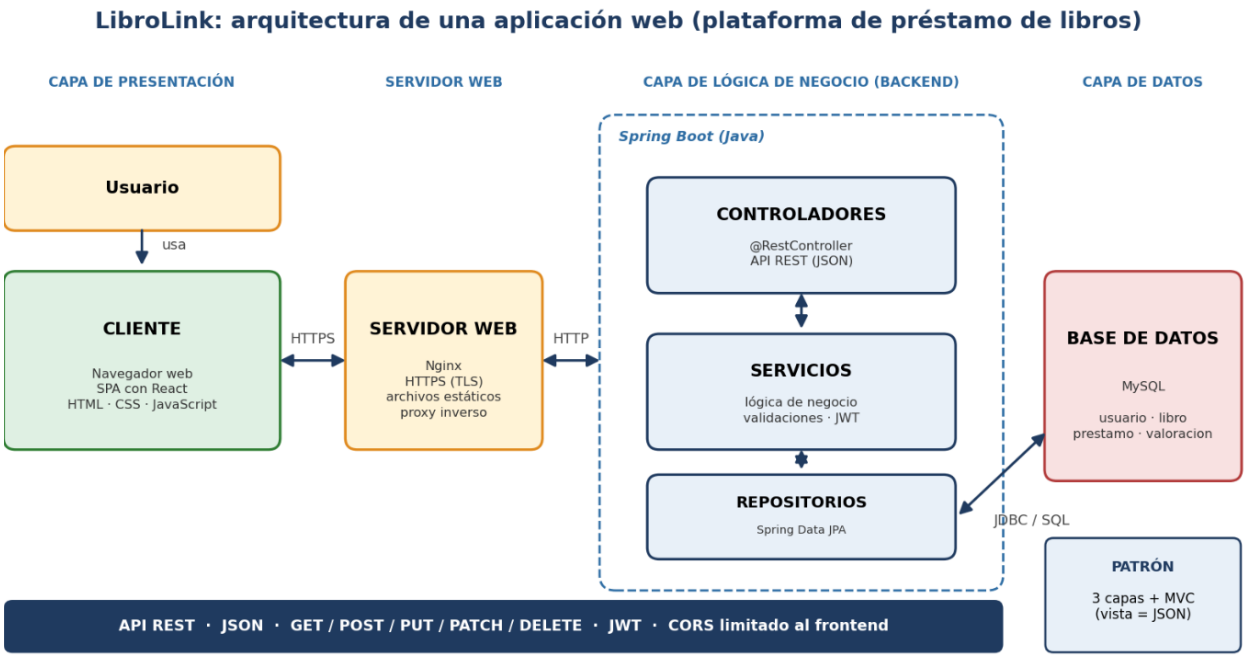
Posible uso de microservicios

- Sí: divide el backend en muchos programas pequeños.
- Cada uno hace una sola tarea: buscar, recomendar, reproducir o cobrar.
- Si uno falla, el resto sigue funcionando.
- Pero es más difícil de organizar.

Tecnologías que podrían emplearse

- Java, Python y JavaScript.
- Docker y Kubernetes.
- Bases de datos como Cassandra y PostgreSQL.

DIAGRAMA



Justificación de las decisiones:

- SPA + API: la web es privada e interactiva y la misma API sirve para una futura app móvil.
- Monolito en capas: el proyecto es pequeño; los microservicios añadirían complejidad innecesaria.
- REST + JWT: sencillo, sin estado y fácil de escalar.
- MySQL: datos muy relacionados y con transacciones. • Nginx: centraliza HTTPS y oculta el backend.

Bibliografía

https://www.daemon4.com/empresa/noticias/arquitectura-cliente-servidor/

https://es.wikipedia.org/wiki/Aplicaci%C3%B3n_m%C3%B3vil

https://www.graphon.com/es/blog/desktop-application

lenovo.com/mx/es/glosario/servidor-web/

https://www.ibm.com/es-es/think/topics/application-server

https://libros.catedu.es/books/bases-de-datos-relacionales-y-lenguaje-sql/page/que-es-una-base-de-datos

http://aws.amazon.com/es/compare/the-difference-between-https-and-http/

https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods/POST

https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods/PUT

https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods/DELETE

https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods/PATCH

https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/capas

https://www.godaddy.com/resources/latam/stories/laravel-que-es-para-que-sirve

https://www.cnac.es/noticias/que-es-spring-mvc/

https://dotnet.microsoft.com/es-es/apps/aspnet/mvc

https://espifreelancer.com/mtv-django.html

https://www.esic.edu/rethink/tecnologia/modelo-vista-controlador-que-es-como-funciona-y-ventajas-c

https://docs.aws.amazon.com/es_es/appsync/latest/devguide/what-is-rest.html

https://www.redhat.com/es/topics/api/what-is-a-rest-api

https://blog.hubspot.es/website/que-es-single-page-application

https://www.reactjs.wiki/que-es-el-server-side-rendering-y-que-ventajas-tiene

https://www.azion.com/es/learning/jamstack/que-es-jamstack/

https://thebridge.tech/blog/desarrollador-front-end-funciones/

https://4geeks.com/es/blog/aprender-a-programar/los-5-frameworks-fundamentales-para-frontend